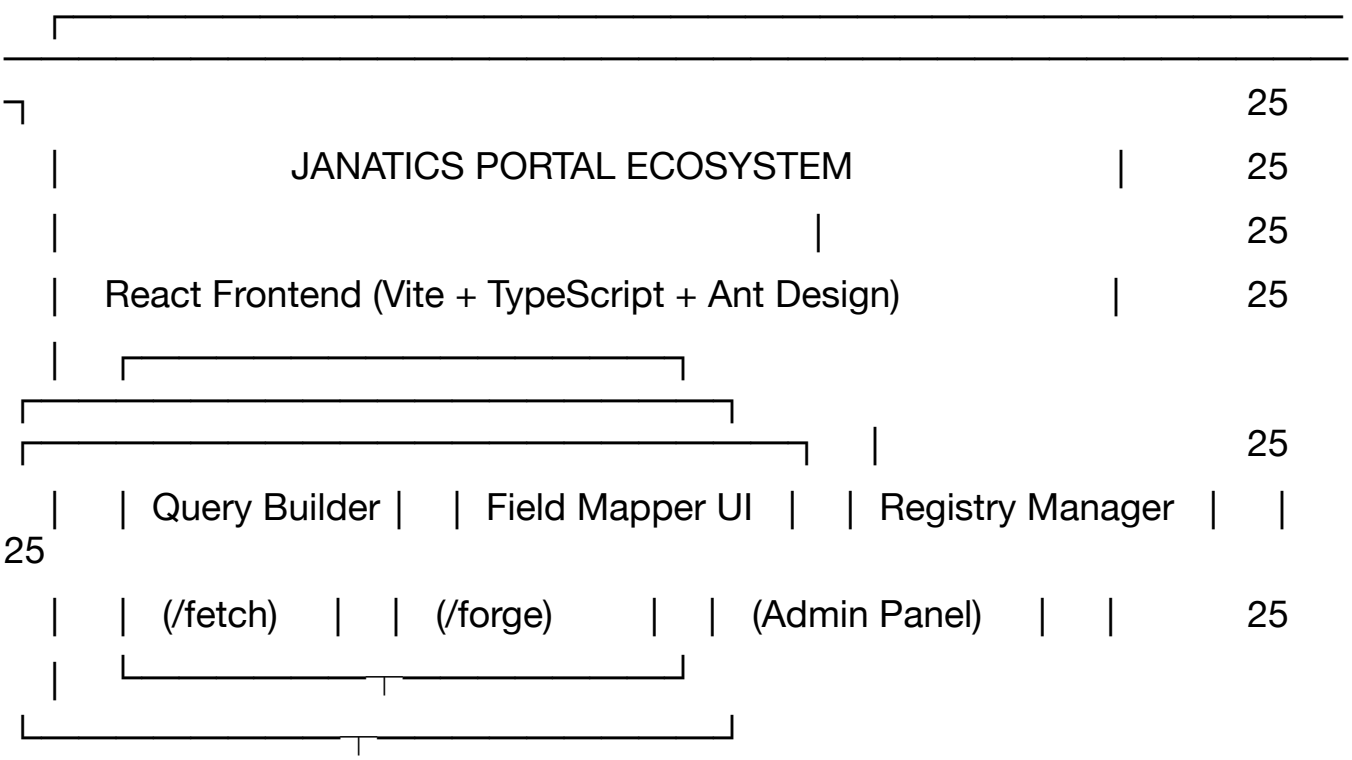
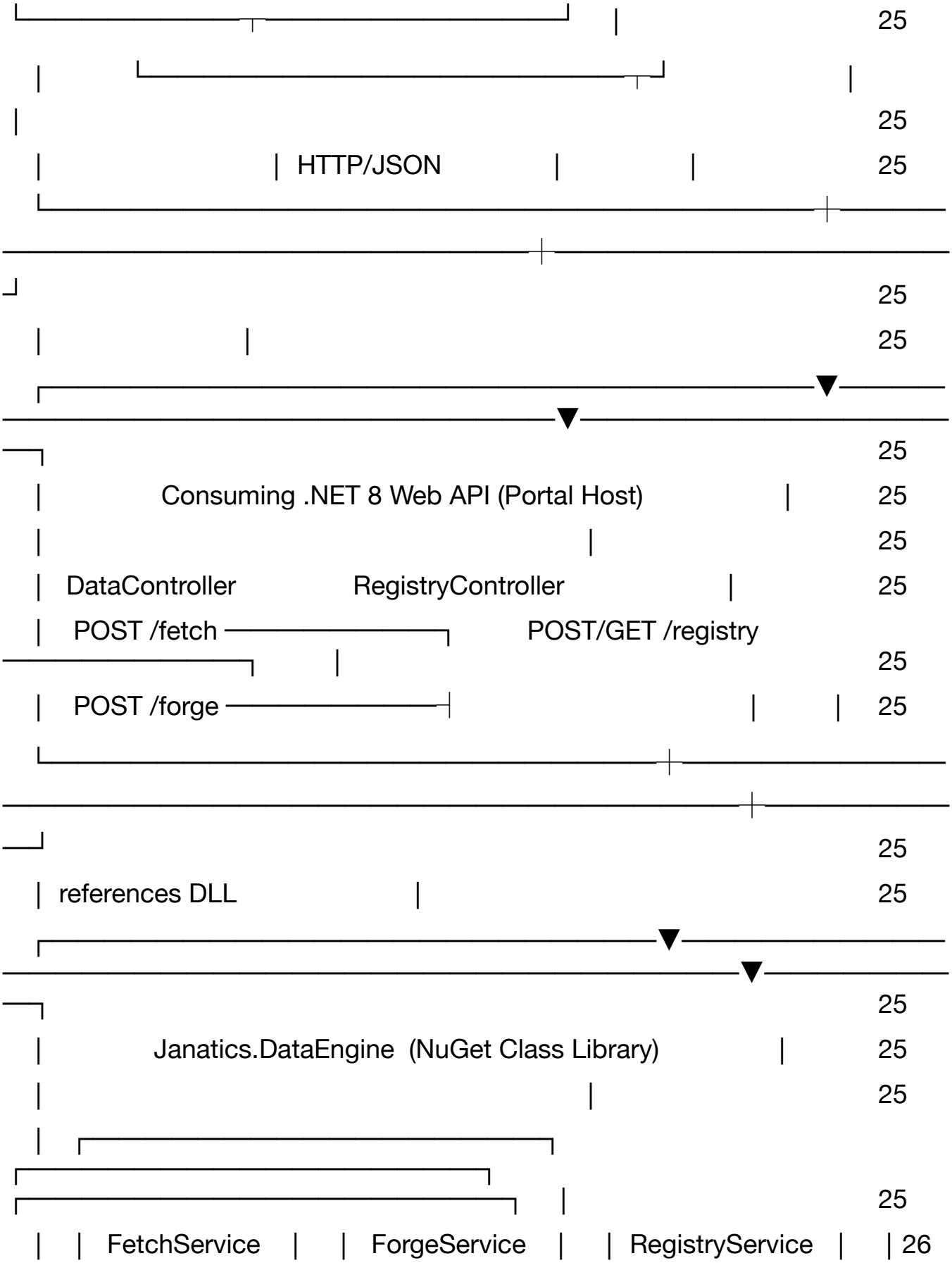
JANATICS DATA ENGINE

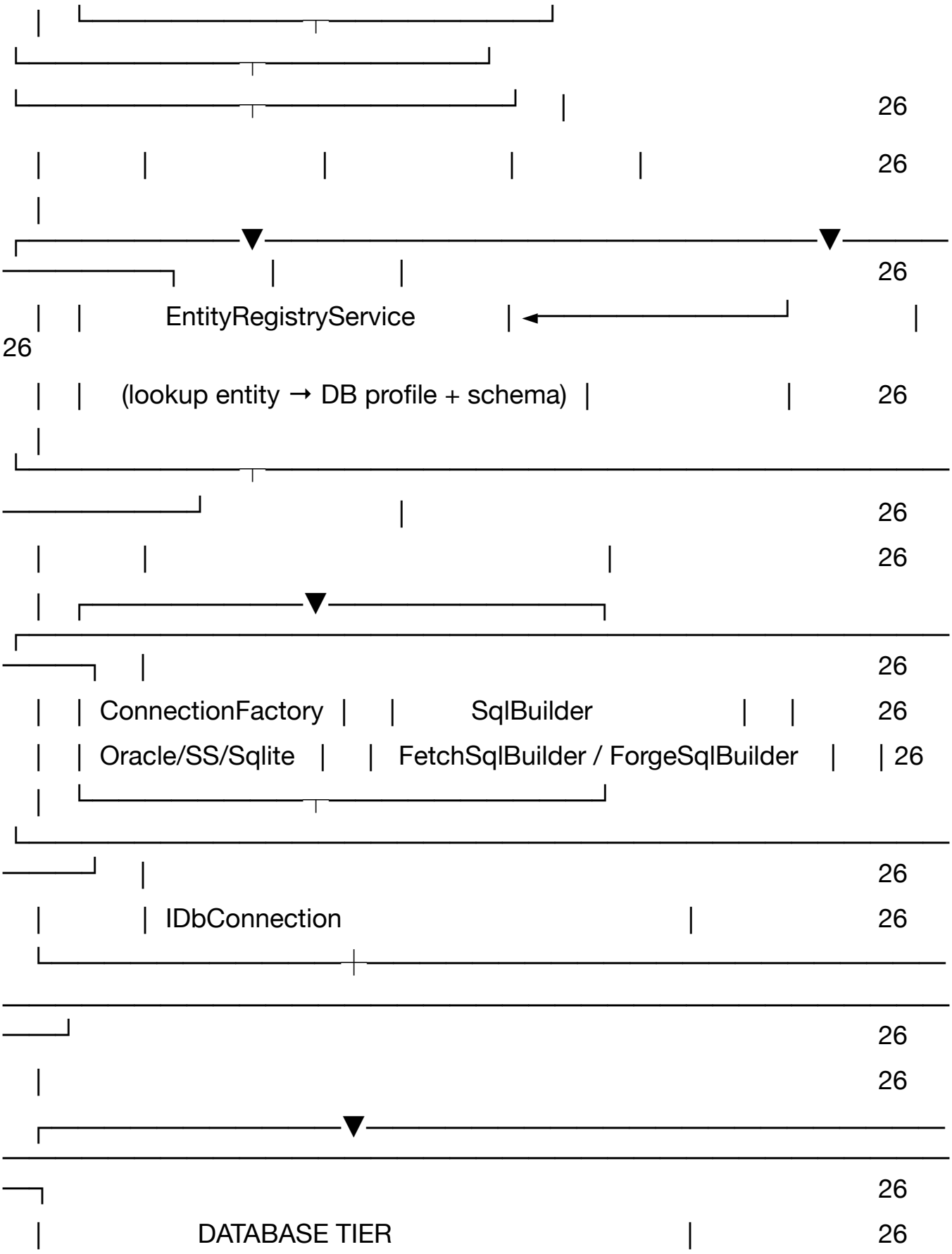
System & Development Design Document

A to Z Architecture Reference · v1.0

Table of Contents




HTTP/JSON
Consuming .NET 8 Web API (Portal Host)
DataController
RegistryController
POST /fetch
POST/GET /registry
POST /forge
references DLL
Janatics.DataEngine (NuGet Class Library)
FetchService
ForgeService
RegistryService


EntityRegistryService
(lookup entity → DB profile + schema)
ConnectionFactory
Oracle/SS/Sqlite
SqlBuilder
FetchSqlBuilder / ForgeSqlBuilder
IDbConnection
DATABASE TIER

```
    |     ├──── SortOptions.cs            ← Column + direction                31
    |     ├──── NodeRequest.cs             ← Child join node (fetch)          31
    |     └──── NodeForge.cs               ← Child write node (forge)         31
    |                                                                         31
    ├──── Registry/                  ← Entity & DB metadata resolution      31
    |     ├──── Models/                                                       31
    |     |     ├──── DbProfile.cs                                          31
    |     |     ├──── EntityMeta.cs                                         31
    |     |     └──── ColumnMeta.cs                                         31
    |     ├──── IEntityRegistryService.cs                                   31
    |     ├──── EntityRegistryService.cs   ← Queries MetaDB, maps entity →
profile                                                                     31
    |     └──── EntityRegistryCache.cs     ← IMemoryCache wrapper, 5-min TTL
31
    |                                                                       31
    ├──── Infrastructure/            ← DB connectivity                      31
    |     ├──── IDbConnectionFactory.cs                                     31
    |     ├──── DbConnectionFactory.cs     ← Resolves provider, builds
IDbConnection                                                               32
    |     ├──── SqlDialect.cs              ← Param prefix, pagination clause, quoting
32
    |     └──── ProviderDetector.cs        ← Auto-sniff from connection string32
    |                                                                       32
    ├──── Builders/                  ← SQL generation (pure, stateless)     32
    |     ├──── FetchSqlBuilder.cs         ← SELECT with filter/sort/page/join 32
```

```
|       ├── Check IMemoryCache (hit → skip DB)                    37
|       └── Cache miss → SELECT from EntityRegistry MetaDB        37
|           Returns: { ProfileId, SchemaName:"JANATICS",          37
|                      PkColumn:"Id", AllowedRoles, IsReadOnly }  37
|                                                                 37
├──▶ [4] WhitelistValidator.Validate(req, entityMeta)             37
|       ├── Entity registered?                                    37
|       ├── Requested columns in ColumnRegistry?                  37
|       ├── Filter columns in ColumnRegistry?                     37
|       └── User role in AllowedRoles?                            37
|           Throws RoleViolationException or SqlBuildException    37
|                                                                 37
├──▶ [5] ConnectionFactory.Create(profileId)                      37
|       ├── Resolve DbProfile from cache/registry                 37
|       ├── Decrypt PasswordHash                                  37
|       └── Return OracleConnection | SqlConnection | SqliteConnection
38
|                                                                 38
├──▶ [6] FetchSqlBuilder.Build(req, entityMeta, dialect)          38
|       ├── Build SELECT clause (all cols or projected)           38
|       ├── Build WHERE clause (AND-chained filter conditions)    38
|       ├── Build LIKE clauses for search term                    38
|       ├── Build JOIN clauses for nodes                          38
|       ├── Build ORDER BY                                        38
|       └── Wrap in provider-specific pagination:                 38
```

```
    |                                                                     40
    ├───▶ [4] ConnectionFactory.Create(profileId)                          40
    |                                                                     40
    ├───▶ [5] Open IDbConnection → BeginTransaction()                      40
    |                                                                     40
    ├───▶ [6] ForgeSqlBuilder.BuildParent(req, entityMeta, dialect)        40
    |         ├──── CREATE:  INSERT INTO JANATICS.PurchaseOrder (...) VALUES
(...)                                                                     40
    |         |          + returning clause → capture new PK              40
    |         ├──── UPDATE:  UPDATE JANATICS.PurchaseOrder SET ... WHERE
Id = :rootId                                                              40
    |         └──── DELETE:  UPDATE JANATICS.PurchaseOrder                40
    |                        SET IsDeleted = 1 WHERE Id = :rootId         40
    |                                                                     40
    ├───▶ [7] Dapper.ExecuteAsync (parent SQL, tx)                         40
    |                                                                     40
    ├───▶ [8] For each node in req.Nodes:                                  40
    |         ├──── Inject parentLink FK = newParentId                    40
    |         ├──── ForgeSqlBuilder.BuildNode(node, dialect)              40
    |         └──── Dapper.ExecuteAsync (node SQL, tx)                    40
    |                                                                     40
    ├───▶ [9] tx.Commit()                                                  40
    |                                                                     40
    ├───▶ [Err] tx.Rollback() on any exception                             40
    |                                                                     41
```

1. Executive Summary

Janatics DataEngine is a **provider-agnostic, registry-driven data access class library** built on .NET 8 that exposes two unified API contracts — `/fetch` and `/forge` — to any consuming .NET application. It eliminates per-entity CRUD boilerplate, supports runtime multi-database routing, and ships a companion React admin suite for registry governance and query composition.

1.1 Problem Statement

- Every new ERP table required a new Repository, Service, Controller, and DTO class — 4 files per entity, hundreds of files across the system
- Switching between Oracle ERP, SQL Server HR, and SQLite Dev required manual connection string management scattered across multiple projects
- No centralised governance of which tables exist in which database, what their primary keys are, or who is authorised to write to them
- Frontend developers had no visual tooling to build queries or inspect registered entities; all payload construction was done by hand
- SQL dialect differences (Oracle ROWNUM/FETCH, SQL Server OFFSET/FETCH, parameter prefix : vs @) were re-implemented ad hoc in every project

1.2 Solution Summary

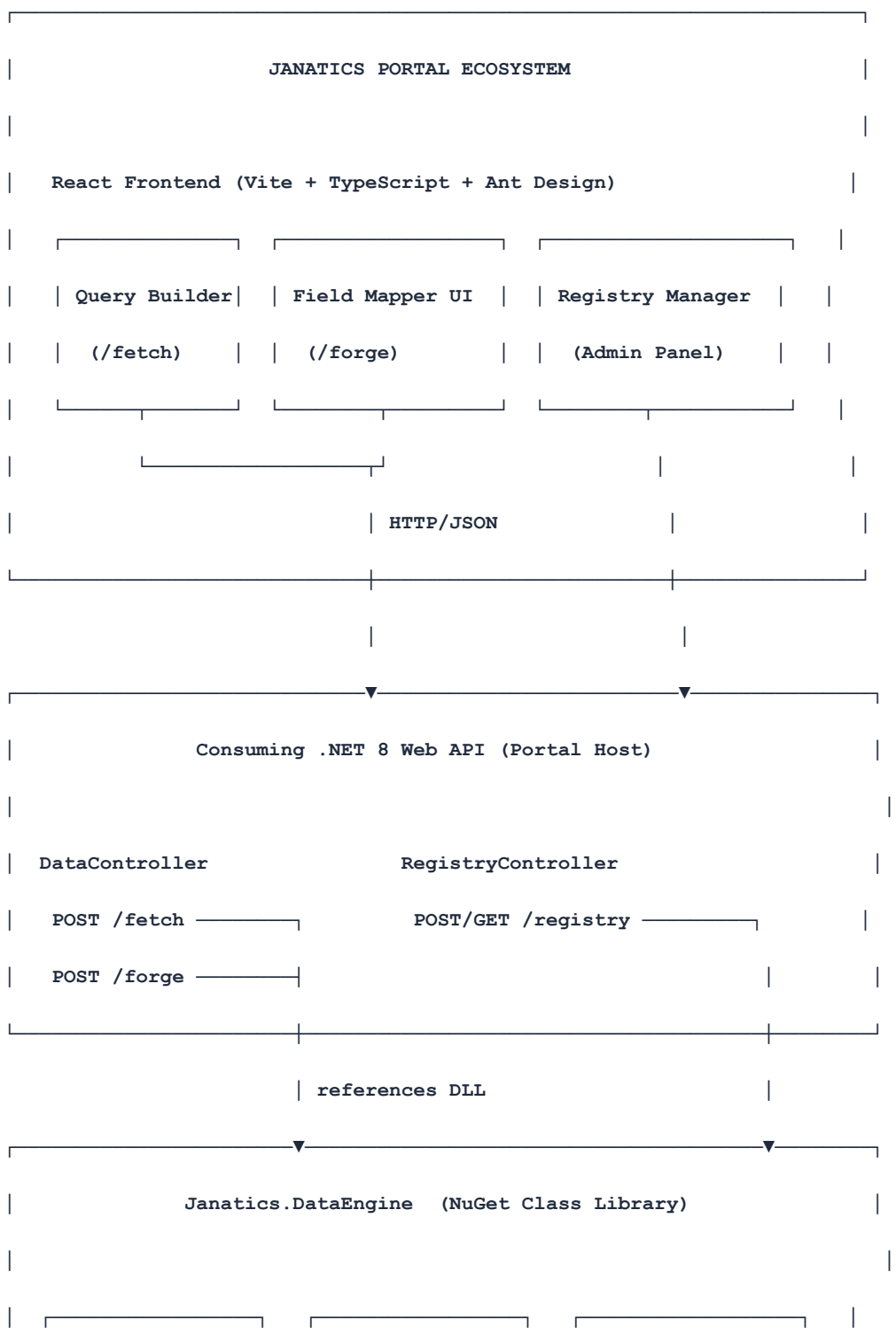
DataEngine packages a **central Entity & DB Registry**, a **runtime connection factory**, a **dialect-aware SQL builder**, and **two service endpoints** into a single NuGet-distributed DLL. Any Janatics portal adds three lines to Program.cs and immediately gains fully parameterised, paginated, multi-database read/write without writing table-specific code.

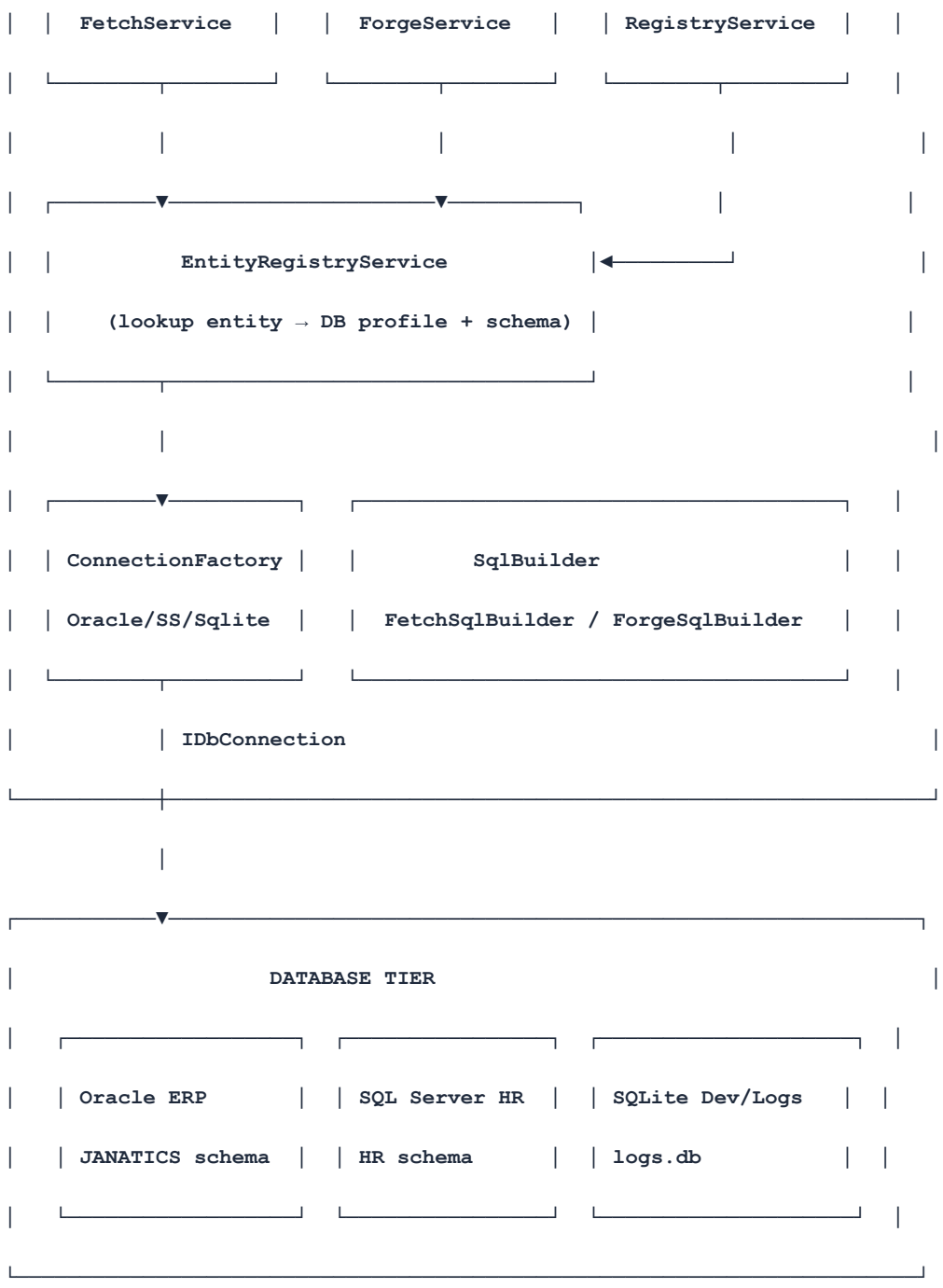
1.3 Business Value

Dimension	Before DataEngine	After DataEngine
New entity onboarding	4 files + review + deploy	1 DB row in registry + UI form
DB migration (Oracle → SQLServer)	Rewrite connection & query layer	Change DatabaseProvider config key
Dev time per CRUD module	2–3 days	< 1 hour (UI config)
SQL injection surface	Per-developer discipline	Centralised whitelist validation
Pagination consistency	Re-implemented per endpoint	Unified FetchRequest contract
Onboarding a new developer	1 week to understand data layer	1 hour to read this document

2. Architecture Overview

2.1 High-Level System Map





2.2 Architectural Principles

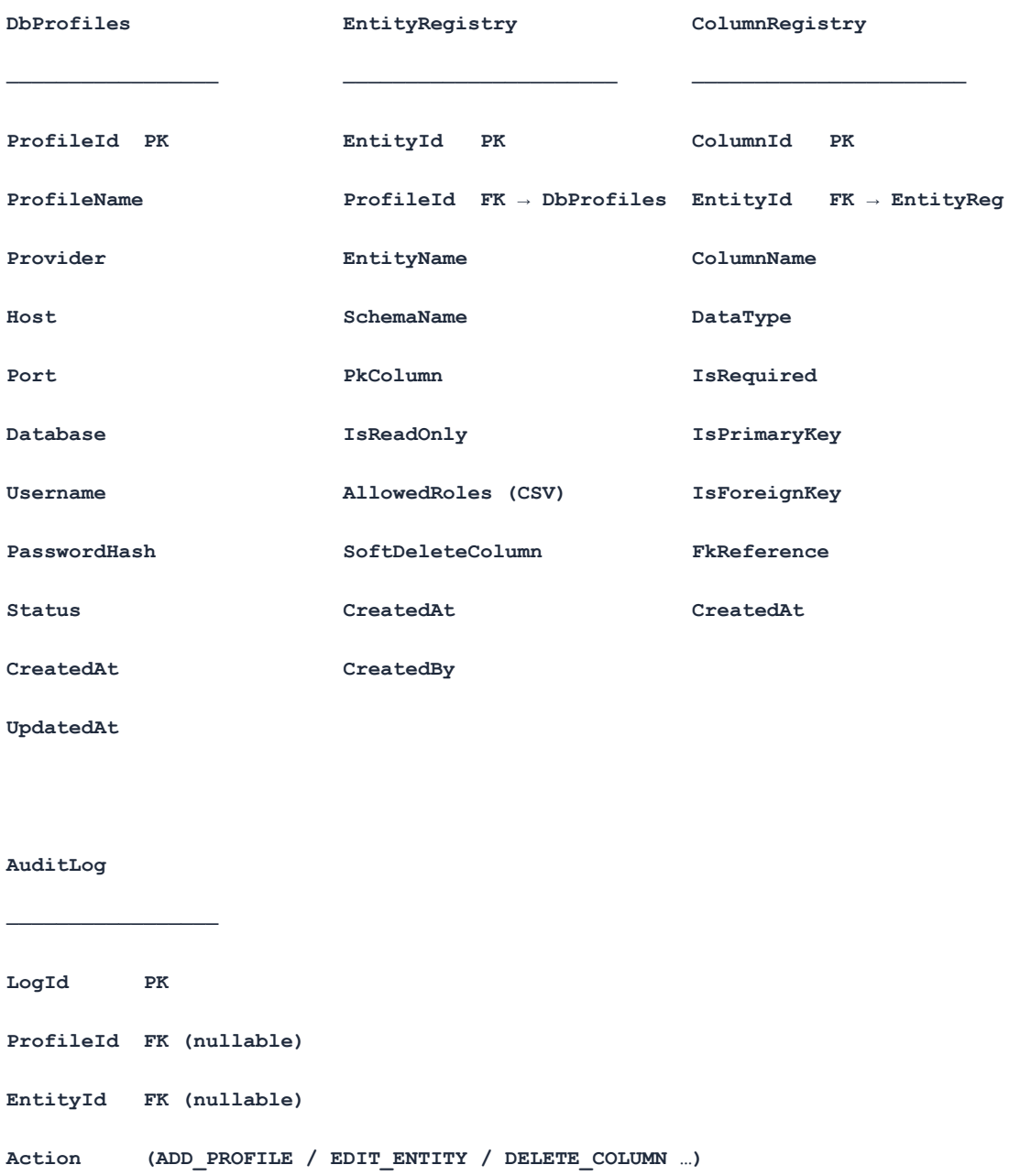
Principle	How DataEngine Implements It
Single Responsibility	Each class does one thing: SqlBuilder only builds SQL, ConnectionFactory only creates connections, RegistryService only resolves metadata
Open/Closed	New DB providers are added by implementing IConnectionFactory — no changes to FetchService or ForgeService

Principle	How DataEngine Implements It
Dependency Inversion	All services depend on interfaces, not concrete implementations; DI container resolves at runtime
Registry Pattern	EntityRegistry is the single source of truth for table metadata; no hardcoded schema knowledge anywhere in the engine
Fail Fast	Whitelist validation against registry rejects unknown entities at request boundary, before any SQL is constructed
Immutable Payloads	FetchRequest and ForgeRequest are sealed records; mutations create new instances
Observability First	Every SQL execution emits structured Serilog events with entity name, provider, duration, and row count

3. Registry Data Model

The Registry is the brain of DataEngine. It is stored in a dedicated metadata database (MetaDB — typically a lightweight SQL Server or SQLite instance) and is independent of any business database.

3.1 Entity-Relationship Overview



```
ChangedBy

ChangedAt

OldValue    JSON snapshot

NewValue    JSON snapshot
```

3.2 DbProfiles Table

Column	Type	Nullable	Description
ProfileId	UNIQUEIDENTIFIER	N	Primary key — GUID
ProfileName	NVARCHAR(100)	N	Human-readable name e.g. OracleERP
Provider	NVARCHAR(20)	N	Oracle | SqlServer | Sqlite
Host	NVARCHAR(255)	Y	Server hostname or IP
Port	INT	Y	Default: Oracle 1521, SQL Server 1433
Database	NVARCHAR(255)	N	DB name or SQLite file path
Username	NVARCHAR(100)	Y	DB login username
PasswordHash	NVARCHAR(500)	Y	AES-256 encrypted, never plaintext
Status	NVARCHAR(20)	N	active | inactive | error
CreatedAt	DATETIME2	N	UTC timestamp
UpdatedAt	DATETIME2	N	UTC timestamp

3.3 EntityRegistry Table

Column	Type	Nullable	Description
EntityId	UNIQUEIDENTIFIER	N	Primary key — GUID
ProfileId	UNIQUEIDENTIFIER	N	FK → DbProfiles.ProfileId
EntityName	NVARCHAR(200)	N	Exact DB table name e.g. PurchaseOrder
SchemaName	NVARCHAR(100)	N	Schema prefix e.g. JANATICS, HR, dbo
PkColumn	NVARCHAR(100)	N	Primary key column name e.g. Id
IsReadOnly	BIT	N	Block INSERT/UPDATE/DELETE if true
AllowedRoles	NVARCHAR(500)	Y	Comma-separated role names
SoftDeleteColumn	NVARCHAR(100)	Y	Column to flip on delete e.g. IsDeleted
SoftDeleteValue	NVARCHAR(50)	Y	Value to set e.g. 1 or true
CreatedAt	DATETIME2	N	UTC timestamp
CreatedBy	NVARCHAR(100)	N	Admin username

3.4 ColumnRegistry Table

Column	Type	Nullable	Description
ColumnId	UNIQUEIDENTIFIER	N	Primary key — GUID
EntityId	UNIQUEIDENTIFIER	N	FK → EntityRegistry.EntityId
ColumnName	NVARCHAR(200)	N	Exact column name in DB
DataType	NVARCHAR(50)	N	string | number | decimal | date | boolean | guid
IsRequired	BIT	N	Validated on forge requests
IsPrimaryKey	BIT	N	Derived from PkColumn but explicit here
IsForeignKey	BIT	N	Whether column references another entity
FkReference	NVARCHAR(300)	Y	EntityName.ColumnName e.g. Vendor.VendorId
CreatedAt	DATETIME2	N	UTC timestamp

4. Janatics.DataEngine — Class Library Structure

4.1 Project Layout

```
Janatics.DataEngine/                ← .NET 8 Class Library project
|
├── Models/                         ← Public API contracts (shared with consumers)
|   ├── FetchRequest.cs             ← /fetch payload model
|   ├── FetchResult.cs              ← Paginated response wrapper
|   ├── ForgeRequest.cs             ← /forge payload model
|   ├── ForgeResult.cs              ← CUD response with affected ID
|   ├── FilterCondition.cs          ← Column + operator + value
|   ├── SearchOptions.cs            ← Full-text search config
|   ├── SortOptions.cs              ← Column + direction
|   ├── NodeRequest.cs              ← Child join node (fetch)
|   └── NodeForge.cs                ← Child write node (forge)
|
├── Registry/                       ← Entity & DB metadata resolution
|   ├── Models/
|   |   ├── DbProfile.cs
|   |   ├── EntityMeta.cs
|   |   └── ColumnMeta.cs
|   ├── IEntityRegistryService.cs
|   ├── EntityRegistryService.cs    ← Queries MetaDB, maps entity → profile
|   └── EntityRegistryCache.cs      ← IMemoryCache wrapper, 5-min TTL
|
├── Infrastructure/                 ← DB connectivity
|   ├── IDbConnectionFactory.cs
```

```
|   ├── DbConnectionFactory.cs      ← Resolves provider, builds IDbConnection
|   ├── SqlDialect.cs               ← Param prefix, pagination clause, quoting
|   └── ProviderDetector.cs         ← Auto-sniff from connection string
|
├── Builders/                       ← SQL generation (pure, stateless)
|   ├── FetchSqlBuilder.cs          ← SELECT with filter/sort/page/join
|   ├── ForgeSqlBuilder.cs          ← INSERT / UPDATE / soft-DELETE
|   └── WhitelistValidator.cs       ← Entity + column name validation
|
├── Services/                       ← Business orchestration
|   ├── IFetchService.cs
|   ├── FetchService.cs             ← Orchestrates /fetch end-to-end
|   ├── IForgeService.cs
|   └── ForgeService.cs             ← Orchestrates /forge end-to-end
|
├── Exceptions/                     ← Typed error hierarchy
|   ├── DataEngineException.cs      ← Base
|   ├── EntityNotFoundException.cs
|   ├── ReadOnlyViolationException.cs
|   ├── RoleViolationException.cs
|   └── SqlBuildException.cs
|
└── Extensions/
    └── ServiceCollectionExtensions.cs  ← AddDataEngine(services, config)
```

4.2 Consumer Integration

Any portal references the DLL and registers it in three lines:

```
// Program.cs
builder.Services.AddDataEngine(builder.Configuration);


// appsettings.json (minimum config)
"DataEngine": {
  "MetaDb": {
    "Provider": "SqlServer",
    "ConnectionString": "Server=metadb;Database=DataEngineRegistry;..."
  },
  "CacheTtlMinutes": 5,
  "EnableAuditLog": true
}


// DataController.cs (minimal — consumer writes this)
[HttpPost("/fetch")]
public Task<IActionResult> Fetch([FromBody] FetchRequest req)
    => _fetchService.ExecuteAsync(req, User);


[HttpPost("/forge")]
public Task<IActionResult> Forge([FromBody] ForgeRequest req)
    => _forgeService.ExecuteAsync(req, User);
```

5. API Contracts

5.1 POST /fetch — Read Operation

The /fetch endpoint accepts a structured query descriptor and returns a paginated, optionally counted result set. It supports column projection, multi-condition filtering, full-text search, sorting, and child table joins — all driven by a single JSON payload.

5.1.1 Request Schema

Field	Type	Required	Default	Description
rootEntity	string	Y	—	Exact entity name from registry
page	int	N	1	Page number (1-based)
pageSize	int	N	25	Rows per page (max 1000)
includeCount	bool	N	true	Whether to run COUNT(*) query
select	string[]	N	all	Column whitelist for projection
filters	array	N	[]	Array of FilterCondition objects
search	object	N	null	SearchOptions: term + columns[]
sort	object	N	null	SortOptions: column + direction
nodes	array	N	[]	Child entities to LEFT JOIN

5.1.2 FilterCondition Object

Operator	SQL Equivalent	Types Supported	Needs value2
eq	= :p0	all	No
neq	!= :p0	all	No
contains	LIKE '%:p0%'	string	No
startsWith	LIKE ':p0%'	string	No
endsWith	LIKE '%:p0'	string	No
gt	> :p0	number, decimal, date	No
gte	>= :p0	number, decimal, date	No
lt	< :p0	number, decimal, date	No
lte	<= :p0	number, decimal, date	No
between	BETWEEN :p0 AND :p1	number, decimal, date	Yes
in	IN (:p0,:p1,...)	string, number	No (CSV value)
isNull	IS NULL	all	No
isNotNull	IS NOT NULL	all	No

5.1.3 Response Schema

Field	Type	Description
success	bool	False if validation or DB error occurred
totalCount	int?	Total rows matching filters (null if includeCount=false)
page	int	Current page number
pageSize	int	Requested page size
totalPages	int?	Calculated ceiling(totalCount / pageSize)
data	object[]	Array of row dictionaries
nodes	object	Map of nodeEntity → array of child rows
error	string?	Human-readable error if success=false
meta	object	Diagnostics: entity, profile, executionMs

5.1.4 Full Example

```
{
  "rootEntity": "PurchaseOrder",
  "page": 2,
  "pageSize": 25,
  "includeCount": true,
  "select": ["Id", "PONumber", "Status", "TotalAmount", "OrderDate"],
  "filters": [
    { "column": "Status", "op": "eq",  "value": "Approved" },
    { "column": "TotalAmount", "op": "gte", "value": 5000 }
  ],
  "search": { "term": "PO-2026", "columns": ["PONumber"] },
  "sort": { "column": "OrderDate", "direction": "desc" },
  "nodes": [
    { "nodeEntity": "PurchaseOrderLine", "parentLink": "PurchaseOrderId" }
  ]
}
```

5.2 POST /forge — Write Operation

The /forge endpoint handles CREATE, UPDATE, and soft-DELETE in a single transactional call. Parent and all child nodes commit atomically; any failure rolls back the entire transaction.

5.2.1 Request Schema

Field	Type	Required	Description
rootEntity	string	Y	Entity name from registry
rootId	string	N	null = CREATE, value = UPDATE or DELETE
operation	string	Y	"create" | "update" | "delete"
content	object	Y	Column → value map for root entity
nodes	NodeForge[]	N	Child records to create/update

5.2.2 NodeForge Object

Field	Type	Required	Description
nodeEntity	string	Y	Child table entity name
parentLink	string	Y	FK column linking child to parent
nodeId	string	N	null = INSERT child, value = UPDATE child
content	object	Y	Column → value map for child record

5.2.3 Operation Behaviour Matrix

rootId	operation	nodeId	Behaviour
null	create	null	INSERT parent → INSERT child with parent FK injected
null	create	"101"	INSERT parent → UPDATE existing child (rare but valid)
"55"	update	null	UPDATE parent → INSERT new child row
"55"	update	"101"	UPDATE parent → UPDATE existing child
"55"	delete	N/A	Sets SoftDeleteColumn = SoftDeleteValue on parent (and optionally children)

6. Runtime Execution Flow

6.1 /fetch End-to-End

```
HTTP POST /fetch  { rootEntity: "PurchaseOrder", filters: [...], page: 2 }
        |
        ▼
  [1] DataController.Fetch(req, User)
        |
        ▼
  [2] FetchService.ExecuteAsync(req, userPrincipal)
        |
        ├──► [3] EntityRegistryService.LookupAsync("PurchaseOrder")
        |         ├── Check IMemoryCache (hit → skip DB)
        |         └── Cache miss → SELECT from EntityRegistry MetaDB
        |               Returns: { ProfileId, SchemaName:"JANATICS",
        |                          PkColumn:"Id", AllowedRoles, IsReadOnly }
        |
        ├──► [4] WhitelistValidator.Validate(req, entityMeta)
        |         ├── Entity registered?
        |         ├── Requested columns in ColumnRegistry?
        |         ├── Filter columns in ColumnRegistry?
        |         └── User role in AllowedRoles?
        |               Throws RoleViolationException or SqlBuildException
        |
        ├──► [5] ConnectionFactory.Create(profileId)
        |         ├── Resolve DbProfile from cache/registry
        |         ├── Decrypt PasswordHash
```

```
        |          └── Return OracleConnection | SqlConnection | SqliteConnection
        |
        ├──► [6] FetchSqlBuilder.Build(req, entityMeta, dialect)
        |          ├── Build SELECT clause (all cols or projected)
        |          ├── Build WHERE clause (AND-chained filter conditions)
        |          ├── Build LIKE clauses for search term
        |          ├── Build JOIN clauses for nodes
        |          ├── Build ORDER BY
        |          └── Wrap in provider-specific pagination:
        |                Oracle:    OFFSET :skip ROWS FETCH NEXT :take ROWS ONLY
        |                SqlServer: OFFSET @skip ROWS FETCH NEXT @take ROWS ONLY
        |                SQLite:    LIMIT :take OFFSET :skip
        |
        ├──► [7] Dapper.QueryAsync(sql, parameters, connection)
        |         Optional: Dapper.ExecuteScalarAsync for COUNT(*)
        |
        └──► [8] Build FetchResult { success, totalCount, page, data, nodes }
                   └── Emit Serilog structured event
                         { entity, profile, rows, executionMs, page }
```

6.2 /forge End-to-End

```
HTTP POST /forge  { rootEntity:"PurchaseOrder", operation:"create", content:{...} }
        |
        ▼
  [1] ForgeService.ExecuteAsync(req, userPrincipal)
        |
        ├──► [2] EntityRegistryService.LookupAsync("PurchaseOrder")
```

```
        |         Same cache/registry lookup as /fetch
        |
        ├──► [3] WhitelistValidator.ValidateForge(req, entityMeta, columnRegistry)
        |         ├── IsReadOnly? → ReadOnlyViolationException
        |         ├── User role in AllowedRoles?
        |         ├── All content keys in ColumnRegistry?
        |         └── Required columns present for CREATE?
        |
        ├──► [4] ConnectionFactory.Create(profileId)
        |
        ├──► [5] Open IDbConnection → BeginTransaction()
        |
        ├──► [6] ForgeSqlBuilder.BuildParent(req, entityMeta, dialect)
        |         ├── CREATE:  INSERT INTO JANATICS.PurchaseOrder (...) VALUES (...)
        |         |            + returning clause → capture new PK
        |         ├── UPDATE:  UPDATE JANATICS.PurchaseOrder SET ... WHERE Id = :rootId
        |         └── DELETE:  UPDATE JANATICS.PurchaseOrder
        |                        SET IsDeleted = 1 WHERE Id = :rootId
        |
        ├──► [7] Dapper.ExecuteAsync (parent SQL, tx)
        |
        ├──► [8] For each node in req.Nodes:
        |         ├── Inject parentLink FK = newParentId
        |         ├── ForgeSqlBuilder.BuildNode(node, dialect)
        |         └── Dapper.ExecuteAsync (node SQL, tx)
        |
        ├──► [9] tx.Commit()
        |
        ├──► [Err] tx.Rollback() on any exception
```

```
        |
        └──► [10] Return ForgeResult { success, entityId, operation, affectedNodes }
```

7. SQL Builder Design

7.1 Dialect Abstraction

SqlDialect is a value type that encapsulates all provider-specific SQL differences. FetchSqlBuilder and ForgeSqlBuilder receive a dialect instance and call its methods — they never branch on provider type directly.

Dialect Concern	Oracle	SQL Server	SQLite
Parameter prefix	:columnName	@columnName	:columnName
Schema qualified name	JANATICS.PurchaseOrder	HR.dbo.Employee	PurchaseOrder (no schema)
Pagination	OFFSET :skip ROWS FETCH NEXT :take ROWS ONLY	OFFSET @skip ROWS FETCH NEXT @take ROWS ONLY	LIMIT :take OFFSET :skip
Returning new ID	RETURNING Id INTO :newId (OracleParameter)	SELECT CAST(SCOPE_IDENTITY() AS NVARCHAR)	SELECT last_insert_rowid()
Boolean literal	1 / 0	1 / 0	1 / 0
String concat	||	+	||
Date functions	SYSDATE, TRUNC()	GETDATE(), CONVERT()	date('now')

7.2 FetchSqlBuilder Logic

FetchSqlBuilder.Build() is a pure function — same inputs always produce same SQL. It operates in five stages:

1. **SELECT clause**: if req.Select is empty → SELECT * else SELECT col1, col2 … (schema-qualified table alias)
2. **WHERE clause**: each FilterCondition mapped to parameterised predicate; parameters named p0, p1 … pN; LIKE values auto-wrapped with %
3. **SEARCH clause**: if search term present, OR-chain across specified columns using provider LOWER() for case-insensitive match
4. **JOIN clause**: each node produces LEFT JOIN {schema}.{nodeEntity} n0 ON n0.{parentLink} = t.{pk}
5. **SORT + PAGINATION**: ORDER BY appended, then dialect.PaginationClause(skip, take) wraps the whole query

Security Note: Column and table names are interpolated into SQL strings after passing WhitelistValidator — they come from the registry, not raw user input. Parameter values are always Dapper parameters, never concatenated.

7.3 ForgeSqlBuilder Logic

```
operation = "create"
  → INSERT INTO {schema}.{table} ({col1},{col2},...)
    VALUES ({p0},{p1},...)
    [+ dialect returning clause]

operation = "update"
  → UPDATE {schema}.{table}
    SET {col1} = {p0}, {col2} = {p1}, ...
    WHERE {pkColumn} = {pId}

operation = "delete" (soft)
  → UPDATE {schema}.{table}
    SET {softDeleteColumn} = {softDeleteValue}
    WHERE {pkColumn} = {pId}

child node (same builder, per node in req.Nodes):
  → parentLink FK auto-injected into content dict
  → same create/update logic applied
```

8. Entity Registry Service & Caching

8.1 Lookup Lifecycle

```
EntityRegistryService.LookupAsync("PurchaseOrder")
  |
  ├── IMemoryCache.TryGetValue("entity:PurchaseOrder")
  |     HIT  → return cached EntityMeta (zero DB calls)
  |     MISS → continue
  |
  ├── SELECT e.*, p.Provider, p.PasswordHash
  |   FROM EntityRegistry e
  |   JOIN DbProfiles p ON p.ProfileId = e.ProfileId
  |   WHERE e.EntityName = @name
  |
  ├── Decrypt PasswordHash → ConnectionString
  |
  ├── Load ColumnRegistry for entity (separate SELECT)
  |
  ├── IMemoryCache.Set("entity:PurchaseOrder", meta,
  |     absoluteExpiration: now + CacheTtlMinutes)
  |
  └── return EntityMeta
```

8.2 Cache Invalidation Strategy

Event	Cache Action
Admin edits entity in Registry Manager UI	PUT /registry/entities/{id}/invalidate → removes cache key
Admin adds column to ColumnRegistry	Same invalidation by entityId

Event	Cache Action
Admin changes DbProfile connection string	Invalidate all entities linked to that profileId
CacheTtlMinutes expires (default 5 min)	Cache auto-evicts; next request repopulates
Application restart	IMemoryCache is in-process; full reload on first request per entity

8.3 ConnectionFactory

ConnectionFactory holds a dictionary of ProfileId → decrypted ConnectionString. It is registered as a singleton and populated at startup from all active DbProfiles. It returns a new IDbConnection per call (connections are not pooled at this layer — ADO.NET connection pool handles physical connection reuse).

Provider	Package	Connection Type
Oracle	Oracle.ManagedDataAccess.Core	OracleConnection
SqlServer	Microsoft.Data.SqlClient	SqlConnection
Sqlite	Microsoft.Data.Sqlite	SqliteConnection

9. Security Design

9.1 Threat Model

Threat	Risk	Mitigation
SQL Injection via entity name	HIGH	Entity and column names validated against registry whitelist before SQL construction; unknown names → 400 immediately
SQL Injection via filter value	HIGH	All filter values passed as Dapper parameters; never concatenated into SQL strings
Unauthorised table access	HIGH	AllowedRoles on EntityRegistry checked against ClaimsPrincipal before any query is built
Write to read-only table	MEDIUM	IsReadOnly flag checked in WhitelistValidator for all forge operations
Connection string exposure	HIGH	Stored AES-256 encrypted in MetaDB; decrypted only at runtime into IDbConnection; never logged or returned to client
Mass data extraction	MEDIUM	pageSize capped at 1000; no full-table export without pagination
Forge with unknown columns	MEDIUM	ColumnRegistry whitelist validates all content keys; unknown columns → 400
Audit trail bypass	LOW	AuditLog appended for every forge operation; no delete path on AuditLog table

9.2 Role Enforcement Flow

```
Request arrives with JWT (or Windows Identity)
  |
  ├── ClaimsPrincipal.GetRoles() → ["user", "hr"]
  |
  ├── EntityMeta.AllowedRoles = ["admin", "hr"]
  |
  ├── Intersection: ["hr"]  → non-empty → ALLOW
  |
  └── Empty intersection → throw RoleViolationException(403)
```

9.3 Connection String Security

- Passwords stored via AES-256-CBC with a machine-level key from environment variable DATA_ENGINE_CRYPTO_KEY
- PasswordHash column contains: base64(IV + ciphertext)
- Key never stored in DB; rotated per environment (dev / staging / prod)
- Connection strings logged with password replaced by ****** in Serilog output

10. Frontend Admin Suite

10.1 Component Map

```
src/
├── pages/
│   ├── RegistryManager.tsx      ← DB Profiles + Entity list + Column panel
│   ├── QueryBuilder.tsx          ← /fetch visual query composer
│   └── FieldMapper.tsx           ← /forge payload builder
│
├── components/
│   ├── registry/
│   │   ├── ProfileCard.tsx
│   │   ├── ProfileForm.tsx
│   │   ├── EntityCard.tsx
│   │   ├── EntityForm.tsx
│   │   └── ColumnPanel.tsx
│   ├── querybuilder/
│   │   ├── FilterRow.tsx
│   │   ├── ColumnSelector.tsx
│   │   ├── SortConfig.tsx
│   │   ├── PaginationConfig.tsx
│   │   └── ResultsTable.tsx
│   └── fieldmapper/
│       ├── ColumnRow.tsx
│       └── ChildSection.tsx
│
├── services/
```

```
|   ├── registryApi.ts          ← CRUD for profiles + entities + columns
|   ├── fetchApi.ts             ← POST /fetch
|   └── forgeApi.ts             ← POST /forge
|
└── types/
    ├── FetchRequest.ts
    ├── ForgeRequest.ts
    └── Registry.ts
```

10.2 Page Descriptions

Page	Primary Role	Writes To
Registry Manager	Admin governance of DB profiles, entities, and columns	MetaDB EntityRegistry + DbProfiles + ColumnRegistry
Query Builder	Developer/analyst tool to build and test /fetch queries visually	/fetch API (read only)
Field Mapper	Form builder that constructs /forge payloads and submits them	/forge API (write)

10.3 Registry Manager Features

- Add, edit, delete DB profiles with auto-generated connection string preview
- Test Connection button — calls dedicated endpoint that attempts a real DB ping
- Auto-Discover tables — queries INFORMATION_SCHEMA / ALL_TABLES and lists unregistered tables for import
- Expandable entity cards showing column registry with type badges and FK annotations
- Column panel with add/remove and full type + constraint configuration
- Audit log viewer — filter by entity, action type, or date range

10.4 Query Builder Features

- Entity selector driven by registry — only registered entities shown
- Column selection with type chips and all-or-none shortcuts
- Multi-condition filter builder with operator dropdown scoped by column data type
- Full-text search with per-column scope selection
- Sort configuration with column selector and ASC/DESC toggle
- Child join (node) configuration with FK column override

- Pagination with preset page-size buttons (10 / 25 / 50 / 100 / 500)
- Live JSON payload preview with copy-to-clipboard
- Execute against real API with inline result table and prev/next navigation

10.5 Field Mapper Features

- INSERT / UPDATE mode toggle — UPDATE reveals Transaction ID input
- Column rows auto-generated from ColumnRegistry — required fields marked, data types respected
- Progress bar showing mapped / total columns
- Child entity sections with FK column config and Record ID for child-level update mode
- Payload preview tab with validation summary (missing required fields listed)
- Submit wired to /forge with success/error state

11. Error Handling & Result Pattern

11.1 Exception Hierarchy

```
DataEngineException  (base)
  ├── EntityNotFoundException      → HTTP 404  (entity not in registry)
  ├── ReadOnlyViolationException   → HTTP 403  (forge on read-only entity)
  ├── RoleViolationException       → HTTP 403  (user role not in AllowedRoles)
  ├── SqlBuildException            → HTTP 400  (unknown column / invalid filter)
  └── ConnectionException          → HTTP 503  (DB unreachable)
```

11.2 Result Types

```
public record FetchResult<T>(
    bool Success,
    IReadOnlyList<T>? Data,
    int? TotalCount,
    int Page,
    int PageSize,
    string? Error,
    FetchMeta Meta
);


public record ForgeResult(
    bool Success,
    string? EntityId,
    string Operation,
    int AffectedNodes,
```

```
    string? Error,

    ForgeMeta Meta

);
```

11.3 Global Exception Middleware

The consuming portal registers DataEngine's exception middleware which maps DataEngineException subtypes to appropriate HTTP status codes and a consistent error envelope:

```
{

  "success": false,

  "error": "Entity 'UnknownTable' is not registered in DataEngine registry.",

  "errorCode": "ENTITY_NOT_FOUND",

  "traceId": "00-4bf5122f-1a2b3c4d-01"

}
```

12. Observability & Logging

12.1 Structured Log Events

Event Name	Level	Key Properties
DataEngine.FetchExecuted	Information	entity, profile, page, pageSize, rowCount, totalCount, executionMs
DataEngine.ForgeExecuted	Information	entity, profile, operation, entityId, nodeCount, executionMs
DataEngine.CacheHit	Debug	entity, cacheKey
DataEngine.CacheMiss	Debug	entity, cacheKey
DataEngine.ValidationFailed	Warning	entity, reason, field
DataEngine.ConnectionFailed	Error	profileName, provider, errorMessage
DataEngine.RollbackExecuted	Error	entity, operation, errorMessage

12.2 Serilog Configuration

```
Log.Logger = new LoggerConfiguration()
    .Enrich.FromLogContext()
    .Enrich.WithProperty("Application", "DataEngine")
    .WriteTo.Console(outputTemplate:
        "[{Timestamp:HH:mm:ss} {Level:u3}] {Message:lj} {Properties}{NewLine}{Exception}")
    .WriteTo.Seq("http://seq.janatics.local")
    .MinimumLevel.Override("Janatics.DataEngine", LogEventLevel.Debug)
    .CreateLogger();
```

12.3 Performance Monitoring

- Every service method uses System.Diagnostics.Stopwatch to capture executionMs
- Thresholds: >500ms logs Warning; >2000ms logs Error with full SQL in Development environments
- MetricsRecorder (optional) emits counters to Application Insights or Prometheus via OpenTelemetry

13. Testing Strategy

13.1 Test Pyramid

Layer	Framework	Scope	Count Target
Unit	xUnit + Moq	SqlBuilder output, WhitelistValidator, DialectMapping, CacheLogic	~120 tests
Integration	xUnit + TestContainers	FetchService + ForgeService against real SQLite DB	~40 tests
Contract	Pact.NET	Consumer-driven contract tests for /fetch and /forge request schema	~20 tests
E2E	Playwright	Registry Manager UI + Query Builder UI against staging environment	~15 tests

13.2 Key Unit Test Cases

- FetchSqlBuilder produces correct Oracle FETCH NEXT pagination clause
- FetchSqlBuilder produces correct OFFSET/FETCH for SQL Server
- FilterCondition "between" produces BETWEEN :p0 AND :p1 with both parameter values
- FilterCondition "contains" wraps value in % on both sides
- WhitelistValidator rejects unknown entity name with EntityNotFoundException
- WhitelistValidator rejects column not in ColumnRegistry with SqlBuildException
- WhitelistValidator rejects user with no matching role with RoleViolationException
- ForgeSqlBuilder for "delete" produces UPDATE ... SET IsDeleted = 1 not DELETE FROM
- EntityRegistryCache returns cached value on second call without hitting DB

13.3 Integration Test Setup

```
public class FetchServiceIntegrationTests : IAsyncLifetime {

    private SqliteConnection _metaConn;

    private SqliteConnection _dataConn;



    public async Task InitializeAsync() {

        _metaConn = new SqliteConnection("Data Source=:memory:");

        await SeedRegistryAsync(_metaConn);   // DbProfile + EntityMeta

        _dataConn = new SqliteConnection("Data Source=:memory:");

        await SeedDataAsync(_dataConn);        // Actual table + rows
```

```
    }


    [Fact]
    public async Task Fetch_WithPagination_ReturnsCorrectPage() { ... }
}
```

14. Deployment & DevOps

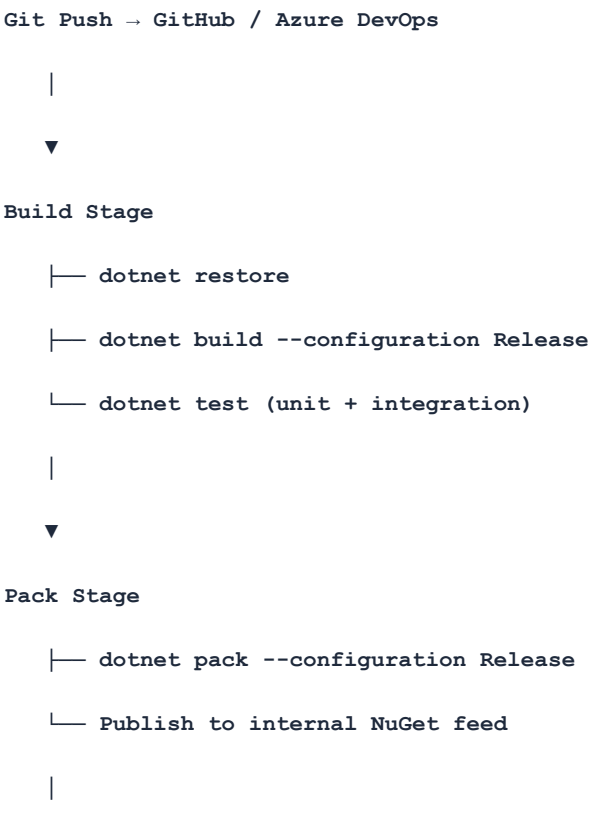
14.1 NuGet Package

- DataEngine is published as Janatics.DataEngine.nupkg to an internal NuGet feed (Azure Artifacts or JFrog)
- Versioning: SemVer 2.0 — MAJOR.MINOR.PATCH (e.g., 1.0.0 → 1.1.0 for new features, 2.0.0 for breaking API changes)
- Consumers pin to a minor version range: <PackageReference Include="Janatics.DataEngine" Version="[1.0,2.0)" />
- CHANGELOG.md maintained per release with migration notes for any breaking changes

14.2 Environment Configuration

Environment	MetaDB	BusinessDB	Cache TTL	Logging Level
Development	SQLite in-memory	SQLite file	0 (disabled)	Debug
Staging	SQL Server	Oracle UAT instance	1 minute	Information
Production	SQL Server HA	Oracle Production	5 minutes	Warning

14.3 CI/CD Pipeline



```
   ▼
Frontend Build
   ├── npm ci
   ├── npm run type-check
   ├── npm run lint
   ├── npm run test (Vitest)
   └── npm run build → dist/
   |
   ▼
Deploy Stage (Staging)
   ├── Run EF migrations on MetaDB
   ├── Deploy API host (IIS / Docker)
   └── Deploy React dist/ → Nginx / Azure Static Web Apps
```

15. Development Roadmap

Phase 1 — Foundation (Weeks 1–3)

Task	Owner	Status
Create Janatics.DataEngine class library project	Backend Dev	Planned
Implement Models: FetchRequest, ForgeRequest, results	Backend Dev	Planned
Implement SqlDialect, ProviderDetector	Backend Dev	Planned
Implement FetchSqlBuilder (SELECT + WHERE + PAGE)	Backend Dev	Planned
Implement ForgeSqlBuilder (INSERT + UPDATE + softDEL)	Backend Dev	Planned
Implement WhitelistValidator	Backend Dev	Planned
Unit tests for all builders (~60 tests)	Backend Dev	Planned

Phase 2 — Registry & Connectivity (Weeks 4–5)

Task	Owner	Status
Design MetaDB schema (DbProfiles, EntityRegistry, ColumnRegistry, AuditLog)	Architect	Planned
Implement EntityRegistryService + cache	Backend Dev	Planned
Implement ConnectionFactory (Oracle + SS + Sqlite)	Backend Dev	Planned
Implement FetchService + ForgeService (orchestration)	Backend Dev	Planned
Implement ServiceCollectionExtensions (AddDataEngine)	Backend Dev	Planned
Integration tests with SQLite MetaDB	Backend Dev	Planned

Phase 3 — Frontend Admin Suite (Weeks 6–8)

Task	Owner	Status
Registry Manager page (profiles + entities + columns)	Frontend Dev	Planned
Registry API service layer (registryApi.ts)	Frontend Dev	Planned
Query Builder page (all filter/sort/page options)	Frontend Dev	Planned
Field Mapper page (INSERT/UPDATE/child nodes)	Frontend Dev	Planned
Connect all UIs to real backend APIs	Fullstack Dev	Planned
Vitest unit tests for all components	Frontend Dev	Planned

Phase 4 — Security & Hardening (Week 9)

Task	Owner	Status
Implement AES-256 password encryption for DbProfiles	Security / Backend	Planned
Role enforcement in WhitelistValidator	Backend Dev	Planned
Global exception middleware	Backend Dev	Planned
Structured Serilog logging in all services	Backend Dev	Planned
Security review of SQL builder output	Architect	Planned

Phase 5 — NuGet & Integration (Week 10)

Task	Owner	Status
Configure dotnet pack and NuGet feed publish	DevOps	Planned
Integrate DataEngine into Janatics PLM portal	Fullstack Dev	Planned
Integrate DataEngine into Budget portal	Fullstack Dev	Planned
E2E Playwright tests (Registry Manager + Query Builder)	QA Dev	Planned
Performance baseline: 10K rows Oracle, P95 < 300ms	Backend Dev	Planned

16. Architecture Decisions & Constraints

16.1 Architecture Decision Records

ADR-001: Dapper over Entity Framework Core

Decision	Use Dapper for all DataEngine SQL execution
Reason	Dynamic SQL built at runtime cannot be modelled with EF Core's static DbSet<T> model; Dapper executes raw parameterised SQL with full provider compatibility
Trade-off	No change tracking, no lazy loading, no migrations — these are handled by consumer applications that may use EF Core for their own models
Alternatives	Micro-ORM: Tortoise ORM (rejected — Oracle support incomplete); Raw ADO.NET (rejected — too verbose for multi-result mapping)

ADR-002: Registry in MetaDB over appsettings.json

Decision	Store EntityRegistry and DbProfiles in a dedicated MetaDB, not in appsettings.json
Reason	Runtime mutability (Registry Manager can add entities without redeployment), UI governability, audit trail, FK relationships between registry tables
Trade-off	Requires MetaDB connection at startup; if MetaDB is down, DataEngine cannot serve requests — mitigated by startup-time full cache warm-up
Migration	appsettings.json fallback supported in v1.0 for development; MetaDB mandatory in staging and production

ADR-003: Soft Delete Only

Decision	"delete" operation in /forge performs UPDATE SET {softDeleteColumn} = {softDeleteValue}, never DELETE FROM
Reason	ERP data is auditable by law in manufacturing context; physical deletes would break foreign key chains and audit trails
Requirement	Every entity registered for delete must have SoftDeleteColumn configured in EntityRegistry

16.2 Known Constraints

- Table name collisions across multiple DBs (same name in Oracle and SQL Server) require unique entity aliases in registry; DataEngine does not auto-resolve collisions
- Stored procedure calls are out of scope for v1.0 — DataEngine executes dynamic SQL only
- Oracle RETURNING INTO requires OracleParameter with output direction — this is encapsulated in OracleDialect.GetNewIdClause() and is the only provider-specific code path in ForgeService
- pageSize is capped at 1000 per request — bulk exports must paginate in the caller
- Complex joins (multi-level grandchild) are deferred to v2.0; v1.0 supports one level of child nodes

17. Glossary

Term	Definition
DataEngine	The Janatics.DataEngine class library — the core DLL described by this document
Registry / MetaDB	The metadata database storing DbProfiles, EntityRegistry, and ColumnRegistry tables
Entity	A registered database table, identified by name and linked to a DbProfile in the registry
DbProfile	A registered database connection (provider + host + credentials) stored in MetaDB
ColumnRegistry	The per-entity column metadata table: name, type, required, PK, FK details
/fetch	The DataEngine read endpoint accepting FetchRequest and returning paginated FetchResult
/forge	The DataEngine write endpoint accepting ForgeRequest and returning ForgeResult
rootEntity	The primary entity targeted by a /fetch or /forge operation
node	A child entity related to the root entity via a foreign key link
parentLink	The FK column on the child entity that references the root entity's PK
rootId	The PK value of the root entity (null signals CREATE; non-null signals UPDATE or DELETE)
operation	Forge operation type: "create", "update", or "delete" (soft)
WhitelistValidator	The DataEngine component that validates entity and column names against the registry
SqlDialect	A value type encapsulating provider-specific SQL syntax differences
FetchSqlBuilder	Pure function that converts FetchRequest to parameterised SELECT SQL
ForgeSqlBuilder	Pure function that converts ForgeRequest to parameterised INSERT/UPDATE SQL
Soft Delete	Marking a record as deleted by updating a flag column rather than removing the row
CacheTtl	Time-to-live for EntityMeta in IMemoryCache (default 5 minutes)
AllowedRoles	Comma-separated role names stored per entity; enforced against user ClaimsPrincipal